

WEEK 2 | PRACTICAL 2

Let Java calculate, compare and convert • expressions and numeric reasoning

Today’s boundary

Extend the typed values from W2-P1. Use arithmetic, relational, logical, assignment and ternary expressions. Do not add control-flow statements or loops yet.

Learning outcomes

Outcome	Student evidence	Status
Use arithmetic and assignment operators	Expressions are written and evaluated	Done
Trace precedence and associativity	Grouping and result are explained	Done
Distinguish integer and decimal division	Both results are predicted	Done
Defend widening/narrowing conversions	Legal conversion and risk are named	Done
Investigate overflow and precision	Observed behaviour is recorded	Done

U1 / S1 — expression lab

Name	Java expression	Inputs / assumptions	Predicted result	Actual result	Concept defended	Status
feeTotal	baseFee + serviceFee	baseFee=1200, serviceFee=150	1350	1350	it's a Basic Arithmetic Addition	Done
courseTotal	courseFee * courseCount	courseFee=800, courseCount=3	2400	2400	it is a Basic Arithmetic multiplication	Done
integerAverage	totalMarks / subjectCount	396 / 5 using int	79	79	it is integer division aproximately	Done
decimalAverage	totalMarks / (double) subjectCount	396 / 5 using cast	79.2	79.2	it is accurate division or precision	Done
withinWeight	weightKg >= MIN && weightKg <= MAX	2.75, 0.1, 20.0	TRUE	TRUE	it Is 'LOGICAL AND' and alos used relational operators	Done
eligible	attendancePercent >= MIN_ATTENDANCE && feePaid	82.5 and true	TRUE	TRUE	here also 'Relational operators', 'Logical AND	Done
message	withinWeight ? "Accepted" : "Review"	withinWeight from previous row	Pass	Pass	Relational operators using Boolean data type	Done

U2 — precedence and associativity trace

Expression	Parenthesised form	Predicted result	Actual result	Why this order?	Status
2 + 3 * 4	2+(3*4)	14	14	Multiplication has a higher precedence than addition.	Done
(2 + 3) * 4	(2+3)*4	20	20	Parentheses () have the high precedence and it is the default order.	Done
100 / 4 + 5	(100/4)+5	30	30	Division has a higher precedence than addition	Done

true false && false	true (false && false)	TRUE	TRUE	Logical AND (&&) has a higher precedence than logical OR ().	Done
total += serviceFee	total = total+serviceFee	1350	1350	Assignment operators (+=) have very low precedence and evaluate right-to-left.	Done

U3 — conversion, overflow and precision clinic

Case	Source -> target	Legal?	Observed value / risk	Repair or safer choice	Status
Widening	int -> long	Yes	Smaller integer fits perfectly inside a larger integer storage	not needed	Done
Widening	int -> double	Yes	it is okay ,but changes layout format (eg:- 5 becomes 5.0)	not needed	Done
Narrowing	double -> int	No	compiler throws error because int stores only integer values not decimal values	use Math.round()	Done
Overflow	int max + 1	Yes	Wraps around to the minimum negative value	Use a larger type like long	Done
Precision	0.1 + 0.2	yes	Floating-point inaccuracy. Evaluates to 0.30000000000000004	using big decimal for precise decimal calculations	Done

P2 compile/run and peer-review evidence

Evidence	Student record	Status
UniFlowExpressionSummary.java	Compile command, run command and copied output	Done
SmartLockerExpressionSummary.java	Compile command, run command and copied output	Done
One expression challenged by a peer	Question, response and revision decision	Done
One conversion/overflow observation	Input, output and plain-language explanation	Done

Runnable programs and expected output

Run all three programs before filling the prediction/actual-result tables. First predict the result from the expression, then compile and run, and finally explain any difference in plain language.

Program 1 — UniFlowExpressionLab.java

```

public class UniFlowExpressionLab {
    public static void main(String[] args) {
        final int PASS_TOTAL = 50;
        final int MIN_EXTERNAL = 30;
        final int GRACE_MARKS = 5;
        final double MIN_ATTENDANCE_PERCENT = 75.0;

        int internalMarks = 20;
        int externalMarks = 31;
        double attendancePercent = 82.5;
        boolean feePaid = true;

        int rawTotal = internalMarks + externalMarks;
        int integerAverage = rawTotal / 2;
        double exactAverage = rawTotal / 2.0;
        boolean meetsTotal = rawTotal >= PASS_TOTAL;
        boolean meetsExternal = externalMarks >= MIN_EXTERNAL;
        boolean eligibleBeforeGrace = meetsTotal && meetsExternal;
        boolean needsReview = attendancePercent < MIN_ATTENDANCE_PERCENT || !feePaid;

        int updatedTotal = rawTotal;
        updatedTotal += GRACE_MARKS;
        String result = (updatedTotal >= PASS_TOTAL && meetsExternal) ? "Pass" : "Fail";

        int attemptCount = 1;
        int attemptsBeforeIncrement = attemptCount;
        attemptCount++;

        System.out.println("=== UniFlow Expression Lab ===");
        System.out.println("Internal marks: " + internalMarks);
        System.out.println("External marks: " + externalMarks);
        System.out.println("Raw total: " + rawTotal);
        System.out.println("Integer average: " + integerAverage);
        System.out.println("Exact average: " + exactAverage);
        System.out.println("Meets total: " + meetsTotal);
        System.out.println("Meets external: " + meetsExternal);
        System.out.println("Eligible before grace: " + eligibleBeforeGrace);
        System.out.println("Needs review: " + needsReview);
        System.out.println("Updated total: " + updatedTotal);
        System.out.println("Result: " + result);
        System.out.println("Attempts before increment: " + attemptsBeforeIncrement);
        System.out.println("Attempts after increment: " + attemptCount);
    }
}

```

Expected output — UniFlowExpressionLab

```

=== UniFlow Expression Lab ===
Internal marks: 20
External marks: 31
Raw total: 51
Integer average: 25
Exact average: 25.5
Meets total: true
Meets external: true
Eligible before grace: true
Needs review: false
Updated total: 56
Result: Pass
Attempts before increment: 1
Attempts after increment: 2

```

```

public class SmartLockerExpressionLab {
    public static void main(String[] args) {
        final double MIN_PARCEL_WEIGHT_KG = 0.1;
        final double MAX_PARCEL_WEIGHT_KG = 20.0;
        final double GRAMS_PER_KG = 1000.0;

        String parcelId = "P-1001";
        char sizeCode = 'M';
        double weightKg = 2.75;
        int intakeSequence = 1001;

        double weightGrams = weightKg * GRAMS_PER_KG;
        int billedGrams = (int) weightGrams;
        boolean positiveWeight = weightKg > 0.0;
        boolean withinWeightLimit = weightKg >= MIN_PARCEL_WEIGHT_KG
            && weightKg <= MAX_PARCEL_WEIGHT_KG;
        boolean supportedSize = sizeCode == 'S' || sizeCode == 'M' || sizeCode == 'L';
        boolean accepted = positiveWeight && withinWeightLimit && supportedSize;
        String message = accepted ? "Accepted" : "Review";
        int batchRemainder = intakeSequence % 4;

        System.out.println("=== Smart-Locker Expression Lab ===");
        System.out.println("Parcel ID: " + parcelId);
        System.out.println("Size code: " + sizeCode);
        System.out.println("Weight (kg): " + weightKg);
        System.out.println("Weight (g): " + weightGrams);
        System.out.println("Billed grams: " + billedGrams);
        System.out.println("Positive weight: " + positiveWeight);
        System.out.println("Within weight limit: " + withinWeightLimit);
        System.out.println("Supported size: " + supportedSize);
        System.out.println("Accepted: " + accepted);
    }
}

```

Expected output — SmartLockerExpressionLab

```

=== Smart-Locker Expression Lab ===
Parcel ID: P-1001
Size code: M
Weight (kg): 2.75
Weight (g): 2750.0
Billed grams: 2750
Positive weight: true
Within weight limit: true
Supported size: true
Accepted: true
Message: Accepted
Batch remainder: 1

```

```
public class ExpressionBoundaryLab {
    public static void main(String[] args) {
        int maximum = Integer.MAX_VALUE;
        int wrapped = maximum + 1;
        long safe = (long) maximum + 1;

        double decimalSum = 0.1 + 0.2;
        int integerDivision = 7 / 2;
        double decimalDivision = 7 / 2.0;
        double lateCast = (double) (7 / 2);

        int permissions = 0b0101;
        int FEE_PAID_BIT = 0b0001;
        int maskedValue = permissions & FEE_PAID_BIT;
        boolean feeBitSet = maskedValue != 0;

        int leftAssociative = 20 - 5 - 3;
        int a, b;
        a = b = 10;

        System.out.println("Wrapped int: " + wrapped);
        System.out.println("Safe long: " + safe);
        System.out.println("Decimal sum: " + decimalSum);
        System.out.println("Integer division: " + integerDivision);
        System.out.println("Decimal division: " + decimalDivision);
        System.out.println("Late cast: " + lateCast);
        System.out.println("Masked value: " + maskedValue);
    }
}
```

Expected output — ExpressionBoundaryLab

Wrapped int: -2147483648
Safe long: 2147483648
Decimal sum: 0.30000000000000004
Integer division: 3
Decimal division: 3.5
Late cast: 3.0
Masked value: 1
Fee bit set: true
Left-associative subtraction: 12
a: 10, b: 10

Compile/run commands: javac UniFlowExpressionLab.java → java UniFlowExpressionLab; javac SmartLockerExpressionLab.java → java SmartLockerExpressionLab; javac ExpressionBoundaryLab.java → java ExpressionBoundaryLab.

Paste actual output into the evidence table above. Do not change a failing source permanently; keep the final version compiling and record experiments in the boundary log.

Submission reminder: include the three .java files, expression trace, conversion/overflow evidence, peer review and Git history. Do not submit .class files.

P2 evidence recording tables

Record exact terminal evidence, not only a summary. Complete the prediction before running; then paste the actual output, first compiler message (or “none”), repair, and file/Git path.

A. Program compile/run evidence

Program file	Folder path	Compile command	Compile result / first error	Run command	Predicted output	Actual output	Status
--------------	-------------	-----------------	------------------------------	-------------	------------------	---------------	--------

UniFlowExpressionLab.java	C:\Documents\PSPJ	javac UniFlowExpressionLab.java	no error	java UniFlowExpressionLab	UniFlow Expression Lab Internal marks: 20 External marks: 31 Raw total: 51 Integer average: 25 Exact average: 25.5 Meets total: true Meets external: true Eligible before grace: true Needs review: false Updated total: 56 Result: Pass Attempts before increment: 1 Attempts after increment: 2	UniFlow Expression Lab === Internal marks: 20 External marks: 31 Raw total: 51 Integer average: 25 Exact average: 25.5 Meets total: true Meets external: true Eligible before grace: true Needs review: false Updated total: 56 Result: Pass Attempts before increment: 1 Attempts after increment: 2	Done
SmartLockerExpressionLab.java	C:\Documents\PSPJ	javac SmartLockerExpressionLab.java	no error	java SmartLockerExpressionLab	Smart-Locker Expression Lab Parcel ID: P-1001 Size code: M Weight (kg): 2.75 Weight (g): 2750.0 Billed grams: 2750 Positive weight: true Within weight limit: true Supported size: true Accepted: true Message: Accepted Batch remainder: 1	Smart-Locker Expression Lab= Parcel ID: P-1001 Size code: M Weight (kg): 2.75 Weight (g): 2750.0 Billed grams: 2750 Positive weight: true Within weight limit: true Supported size: true Accepted: true Message: Accepted Batch remainder: 1	Done
ExpressionBoundaryLab.java	C:\Documents\PSPJ	javac ExpressionBoundaryLab.java	no error	java ExpressionBoundaryLab	Wrapped int: -2147483648 Safe long: 2147483648 Decimal sum: 0.30000000000000004 Integer division: 3 Decimal division: 3.5 Late cast: 3.0 Masked value: 1 Fee bit set: true Left-associative subtraction: 12 a: 10, b: 10	Wrapped int: -2147483648 Safe long: 2147483648 Decimal sum: 0.30000000000000004 Integer division: 3 Decimal division: 3.5 Late cast: 3.0 Masked value: 1 Fee bit set: true Left-associative subtraction: 12 a: 10, b: 10	Done

B. Expression and boundary observations						
Case / expression	Prediction	Actual value	Why Java produced this	Risk or invariant	Repair / safer choice	Status
2 + 3 * 4	14	14	Operator precedence: multiplication (*) executes before addition (+).	code error in readability	use parantheses : 2+(3 * 4)	Done
396 / 5 using int	79	79	intrger division drops fractional or decimal part	the loss of decimal part or precision	using double	Done
396 / (double) 5	79.2	79.2	Explicit casting to double forces floating-point division	no risk and Standard binary floating-point representation	no repair needed	Done
int max + 1	-2147483648	-2147483648	Two's complement integer storage wraps maximum positive to minimum negative	integer overflow causing severe math/logic errors.	we have to use long data type	Done
0.1 + 0.2	0.3	0.3	it floating-point formatting cannot perfectly store base-10 decimals	loss of precision and extra decimals after the output	using big Decimals is a safer choice	Done
withinWeight ? "Accepted" : "Review"	"Accepted"	"Accepted"	conditional operator checks the true values and produ true conditione statment	in complex nesting readability reduced	keeping standard conditions of if-esle blocks	Done

C. Peer review and Git evidence

Evidencee item	Student record	Partner / commit / file path	Status
Peer question and response			Not started
One prediction changed after the run			Not started
git status output / clean-folder check			Not started
git diff review / files checked			Not started
Commit hash and GitHub URL			Not started

After completing the tables, update the P2 status in Overview. Attach the copied Sheet plus the three .java files and evidence folder, or submit the GitHub link requested by the instructor.